LAPORAN TUGAS BESAR

IF4031 Arsitektur Aplikasi Terdistribusi

Milestone 1

Sistem Terdistribusi Dasar

Platform Koordinasi Kebencanaan BNPB



DikejarDeadline

Refki Alfarizi	13523002
M Hazim R Prajoda	13523009
Adhimas Aryo Bimo	13523052

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

1 Deskripsi Umum Sistem **1**

2 Arsitektur Sistem **2**

3 Teknologi, Asumsi, dan Cara Menjalankan **5**

4 Problem 1: Arsitektur & Interoperabilitas Data **8**

5 Problem 2: Concurrency, Availability, & Graceful Degradation **17**

6 Problem 3: Service-to-Service Authentication & Trust **23**

7 Problem 4: Arsitektur Microservice & Flexible Storage **31**

8 Problem 5: Fan-out Event Distribution/Pub-sub **35**

9 Kesimpulan **45**

10 Pembagian Tugas **46**

Referensi **47**

A Penelusuran Kode dan Bukti **48**

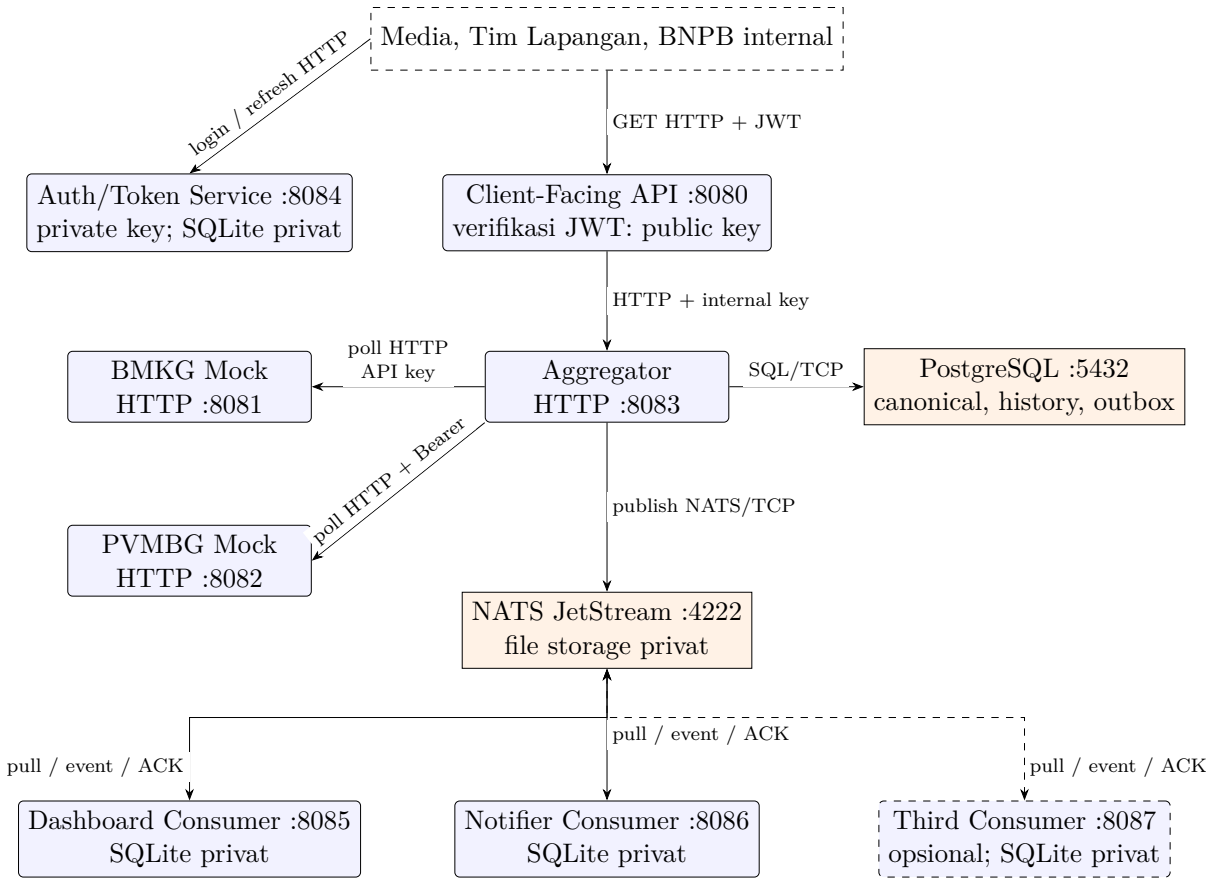
Deskripsi Umum Sistem

Platform ini mengumpulkan data kebencanaan dari simulasi BMKG dan PVMBG agar client dapat membacanya melalui satu API. Kedua sumber memiliki format dan cara autentikasi yang berbeda. Aggregator menyamakan data tersebut menjadi `HazardEvent`, kemudian menyimpannya di Canonical Store, yaitu penyimpanan utama untuk data yang sudah mengikuti format bersama. Dengan demikian, client tidak perlu melakukan pemetaan ulang untuk setiap sumber.

Data yang diterima client disesuaikan dengan identitasnya. Media memperoleh ringkasan, sedangkan Tim Lapangan dan BNPB internal dapat membaca data lengkap. Selain tersedia melalui API, setiap event baru atau revisi event diteruskan ke NATS JetStream. Dashboard Consumer dan Notifier Consumer kemudian mengolahnya secara terpisah, sehingga gangguan pada salah satu consumer tidak menghentikan consumer lainnya.

Milestone 1 berfokus pada alur pengambilan, pembacaan, dan distribusi data tersebut. Implementasinya mencakup autentikasi sumber, polling, pemetaan, penyimpanan, pembatasan akses client, serta distribusi event ke beberapa consumer. Sesuai spesifikasi M1 dan dokumen terpusat, seluruh data pada tahap ini bersifat sintetis [1, 2]. Sistem baru menangani `SEISMIC` dan `VOLCANIC`; penerimaan laporan lapangan dan pengiriman SMS sungguhan belum termasuk dalam implementasi.

Arsitektur Sistem



Gambar 2.1: Setiap kotak berwarna adalah satu container. SQLite berada di volume privat service pemiliknya. Panah menunjukkan komunikasi jaringan dan alur event. Consumer memakai NATS/TCP untuk pull, delivery, dan ACK. Respons HTTP dan SQL kembali melalui koneksi yang sama. Port adalah default proyek.

2.1. Alur data dan batas service

Alur dimulai ketika Aggregator melakukan polling ke BMKG dan PVMBG. Masing-masing sumber memiliki loop sendiri agar sumber yang lambat tidak menunda pengambilan data dari sumber lain. Setelah respons dipetakan menjadi `HazardEvent`, Aggregator menyimpan data terbaru, snapshot revisi, dan pesan outbox dalam satu transaksi PostgreSQL. Outbox merupakan daftar pesan yang sudah menjadi bagian dari data tersimpan, tetapi masih perlu dikirim ke broker. Relay membacanya dan melakukan publish ke JetStream, termasuk mencoba kembali jika publish gagal.

Ada dua jalur untuk menggunakan data tersebut. Pada jalur baca, client memperoleh JWT dari Auth, lalu mengirimkannya ke Client-Facing API. API memeriksa signature dan scope secara lokal sebelum membaca data melalui API internal Aggregator. Pada jalur event, Dashboard dan Notifier mengambil pesan dari JetStream, mengolahnya, ke-

mudian menyimpan hasil pada database masing-masing. Notifier hanya mencatat alert lokal sebagai simulasi notifikasi pada M1.

2.2. Alasan pembagian service

Kontrak BNPB pada spesifikasi M1 mencantumkan Aggregator, Client-Facing API, Auth/Token Service, dan sedikitnya dua Event Consumer. Ketentuan implementasi juga mewajibkan container terpisah serta satu pemilik untuk setiap storage [1]. Pembagian tanggung jawab berikut menerapkan ketentuan tersebut dan membatasi bagian yang harus berubah bersama.

Aggregator menangani polling, pemetaan, dan transaksi Canonical Store, sedangkan Client API memeriksa izin client dan menyajikan data. Dengan pemisahan ini, perubahan login atau antarmuka client dapat dilakukan tanpa membangun ulang ingest. Client API tetap membaca melalui HTTP Aggregator sesuai aturan ownership. REST/JSON dipilih karena bentuk data yang sama dapat digunakan pada jalur baca dan mudah diperiksa dengan curl. Konsekuensinya, pembacaan membutuhkan satu panggilan jaringan tambahan dan tetap bergantung pada Aggregator.

Auth menangani pemeriksaan credential client, penerbitan token, dan sesi refresh. Client API memverifikasi token serta menegakkan hak akses, sehingga ia cukup memiliki public key; private key tetap berada di Auth. Menggabungkan keduanya akan mengurangi jumlah proses, tetapi mengikat deploy login dan pembacaan data dalam satu unit serta menempatkan key penandatanganan pada proses yang melayani data. Dengan verifikasi JWT lokal, gangguan Auth tidak menghentikan penggunaan access token yang masih valid. Login dan refresh tetap memerlukan Auth.

Polling BMKG dan PVMBG tetap berada dalam satu Aggregator karena keduanya membentuk HazardEvent dan menggunakan Canonical Store yang sama. Loop serta pool koneksi dipisahkan untuk mengisolasi pekerjaan sumber tanpa menambah service ingest per instansi. Relay outbox juga berjalan sebagai worker di Aggregator, sehingga pembacaan outbox tetap dilakukan oleh pemilik storage. Batas ini mengurangi jumlah antarmuka yang harus dikelola, tetapi polling, pemetaan, dan relay masih ikut berhenti jika container Aggregator mati.

Dashboard dan Notifier dipisahkan karena hasil olahannya berbeda: Dashboard menyimpan proyeksi data terbaru, sedangkan Notifier mencatat alert. Satu proses dengan dua handler dapat menjalankan kedua fungsi, tetapi penghentian proses tersebut akan menghentikan keduanya. Container dan durable yang terpisah memungkinkan setiap consumer berhenti, pulih, atau berubah tanpa mengganti consumer lain.

2.3. Jaringan dan kepemilikan storage

Canonical Store hanya dapat diakses langsung oleh Aggregator; Client API mengaksesnya melalui HTTP, sedangkan consumer menerima event melalui broker. PostgreSQL dan NATS tidak membuka port ke host. Auth dan setiap consumer juga memiliki volume SQLite sendiri, sehingga mereka tidak saling membaca database lokal.

Empat jaringan Compose dipilih untuk membatasi hubungan sesuai jalur komunikasinya. Dibanding satu jaringan untuk seluruh container, konfigurasi ini mengurangi jalur akses

yang tidak diperlukan, dengan konsekuensi pengaturan jaringan menjadi lebih banyak. Pembatasan tersebut melengkapi autentikasi pada setiap antarmuka.

Jaringan	Anggota dan fungsi
`source-net`	Dua mock dan Aggregator; polling sumber.
`api-net`	Client API, Auth, Aggregator; jalur API internal.
`store-net`	Hanya Aggregator dan PostgreSQL; jaringan internal.
`event-net`	Aggregator, NATS, dan consumer; jaringan internal.

Tabel 2.1: Pembagian jaringan dalam `docker-compose.yml`.

Antarservice berkomunikasi melalui antarmuka jaringan. Paket bersama dipakai untuk kontrak pesan dan utilitas teknis; aturan pemetaan, otorisasi, serta pengolahan event tetap berada pada service yang bertanggung jawab.

Demo console disediakan sebagai alat operator untuk menjalankan command HTTP dan Docker serta melihat output terminal. Alat ini berbeda dari Dashboard Consumer yang mengolah event. Seluruh service dapat dijalankan dan diuji melalui CLI tanpa membuka demo console.

Teknologi, Asumsi, dan Cara Menjalankan

3.1. Pemilihan teknologi

Teknologi dipilih untuk mendukung polling terpisah, perubahan atribut data, dan distribusi event ke consumer independen pada M1.

Komponen / teknologi	Alasan pemilihan
Service aplikasi: Go	Goroutine yang ringan mendukung worker polling terpisah dan request concurrent pada P2 [6]. Aplikasi dapat dibangun menjadi binary statis dan dijalankan dalam image `scratch` tanpa interpreter. Hal ini memudahkan pengemasan serta deployment tiap service.
Canonical Store: PostgreSQL 16, JSONB	Kolom bertipe menjaga identitas dan constraint, sedangkan JSONB menampung atribut tambahan [3]. Dibanding schema yang menjadikan setiap atribut sebagai kolom, JSONB menerima field baru tanpa migrasi tabel. Data utama, riwayat revisi, dan outbox juga dapat disimpan dalam satu transaksi PostgreSQL.
Broker: NATS 2.10 JetStream	NATS ringan dari sisi deployment: routing dan JetStream tersedia dalam satu executable `nats-server` tanpa runtime eksternal [4]. Satu stream dan durable consumer menyediakan fan-out, backlog, ACK, serta replay P5. RabbitMQ fan-out memerlukan queue dan binding per consumer; replay setelah ACK pada queue biasa memerlukan mekanisme seperti RabbitMQ Streams [8]. Kafka juga mendukung replay, tetapi pengelolaan partition belum diperlukan untuk pembagian beban M1 [7]. JetStream dipilih karena deployment dan konfigurasi integrasinya sederhana untuk beberapa consumer pada satu host.
State lokal: SQLite	Penyimpanan di memory akan kehilangan sesi refresh dan inbox saat restart. SQLite menyediakan persistence dan transaksi sebagai library di dalam aplikasi, tanpa database server tambahan [9]. Pilihan ini cukup untuk satu instance per service pada M1; jika beberapa replica perlu berbagi state dan menulis bersamaan, database server seperti PostgreSQL lebih sesuai.
Orchestration: Docker Compose	M1 dijalankan pada satu host. Compose menyatukan konfigurasi container, jaringan, dan volume dalam satu berkas [10]. Dibanding Kubernetes, kebutuhan saat ini dapat dipenuhi tanpa menyiapkan cluster. Opsi `-no-deps` mendukung penggantian satu service secara terpisah pada P4.
Kontrak: JSON Schema	Berkas schema mendefinisikan aturan HazardEvent, ringkasan Media, dan envelope event untuk digunakan kembali dalam test. Dibanding pemeriksaan field secara manual, aturan tipe dan field wajib lebih mudah dijaga konsisten ketika kontrak berubah [11].

Tabel 3.1: Alasan pemilihan teknologi untuk kebutuhan M1.

PostgreSQL dan NATS digunakan dalam konfigurasi satu node pada M1. Konfigurasi ini sesuai untuk eksperimen lokal, tetapi belum menyediakan failover jika host atau storage utama berhenti.

3.2. Asumsi implementasi

Rancangan dan pengujian menggunakan asumsi berikut. Asumsi ini membatasi cara hasil uji ditafsirkan, terutama ketika sistem dikembangkan untuk volume atau jumlah instance yang lebih besar.

1. Kedua sumber merupakan mock yang memulai sedikitnya 20 record dan menambah record setiap 10 detik. Lokasi gunung berasal dari fixture sintetis Tamriel, sehingga tidak mewakili lokasi operasional PVMBG.
2. Pasangan `(source, source_ref_id)` tetap menjadi identitas data sumber. Jika isi record berubah, sistem membuat revisi dari hazard yang sama, bukan hazard baru.
3. Polling dilakukan setiap 5 detik dengan mengambil snapshot sumber yang dibatasi 1 MiB. Cara ini memungkinkan perubahan pada record lama ikut ditemukan, dengan biaya transfer yang meningkat seiring pertumbuhan data.
4. Perubahan schema yang didukung adalah penambahan atribut. Perubahan nama, tipe, atau makna field wajib perlu ditangani melalui perubahan mapper. Timestamp harus memiliki timezone dan dinormalisasi ke UTC.
5. Media memiliki scope `hazards:summary`, sedangkan Tim Lapangan dan BNPB internal memiliki `hazards:summary hazards:raw`. Access token berlaku 60 detik dan sesi refresh memiliki batas umur 24 jam.
6. Waktu antarservice diusahakan sinkron. Latency diukur dengan timer monotonic agar perubahan jam host tidak mengubah durasi pengukuran, tetapi expiry JWT dan umur data tetap bergantung pada wall clock.
7. Setiap aplikasi, PostgreSQL, dan NATS berjalan sebagai satu instance. Karena belum ada replica untuk failover, hasil M1 tidak menyatakan bahwa sistem tetap tersedia ketika host atau storage utama mati.
8. Secret disediakan melalui environment dan file key lokal. Komunikasi HTTP internal belum menggunakan TLS; jaringan Compose membatasi hubungan antarcontainer, tetapi administrator host tetap dapat mengakses container dan volume.

3.3. Cara menjalankan sistem

Persiapan dilakukan dari root repo. Salin `.env.example` menjadi `.env`, isi credential yang berbeda untuk setiap sumber dan client, lalu buat pasangan key RSA di `.secrets/`. Script berikut memasukkan private key dan public key ke dua volume terpisah sebelum stack dijalankan:

```
python3 scripts/setup-jwt-keys.py
docker compose up --build -d --wait
```

Opsi `-wait` menunggu service mencapai kondisi siap berdasarkan healthcheck. Setelah stack berjalan, seluruh skenario integrasi dapat dijalankan dengan:

```
python3 scripts/verify-integration.py --scenario all \
  --output docs/evidence/integration-trial.json
```

Script tersebut menjalankan pengujian, termasuk mengubah schema, memicu outage, dan menghentikan consumer. Oleh karena itu, pengujian dilakukan pada stack lokal yang digunakan khusus untuk eksperimen. Langkah persiapan lengkap tersedia di README root dan `scripts/README.md`.

Credential service dimuat dari `.env` ke environment shell sebelum command pengujian dijalankan:

```
set -a
. ./.env
set +a
```

Access token client diperoleh melalui login ke Auth menggunakan identitas dan secret client masing-masing.

Consumer ketiga dapat dijalankan melalui profile `demo`. Setelah pengujian selesai, gunakan `docker compose -profile demo stop` untuk menghentikan container tanpa menghapus data pada volume.

3.4. Lingkungan pengujian

Pengujian menggunakan port host 8280–8284 untuk API, BMKG, PVMBG, Aggregator, dan Auth. Seed, interval polling, TTL, dan batas koneksi tetap memakai nilai default M1. Khusus pengujian P2, delay PVMBG diubah sementara menjadi 3000 ms.

Load test pada 5 Oktober 2026 berjalan di WSL2 Linux dengan 12 CPU logical dan RAM sekitar 15,6 GiB, menggunakan Docker 27.0.3 serta Compose 2.28.1. Image aplikasi dibangun dengan Go 1.23, sedangkan test kode lokal dijalankan dengan Go 1.26.4. Rincian konfigurasi yang tidak mengandung secret tersimpan di `evidence/environment.json`.

Problem 1: Arsitektur & Interoperabilitas Data

4.1. Mekanisme pemetaan

BMKG dan PVMBG menyampaikan informasi dengan field yang berbeda. Perbedaan ini ditangani oleh mapper di Aggregator, sehingga API dan consumer menerima bentuk `HazardEvent` yang sama. Identitas, jenis bahaya, severity, lokasi, dan waktu menjadi field canonical, sedangkan informasi khusus sumber disimpan dalam `attributes`. Tabel berikut menunjukkan hubungan antara field sumber dan hasil pemetaannya.

Field canonical	BMKG	PVMBG
`source_ref_id`	`event_id`	`report_id`
`hazard_type`	`SEISMIC`	`VOLCANIC`
`area_name`	`region_name`	Lookup `volcano_id`
Koordinat	`epicenter_lat/lon`	Lookup fixture gunung
`occurred_at`	`occurred_at`	`reported_at`
`severity`	Magnitude atau peringatan tsunami terkait	Normal, Waspada, Siaga, Awas dipetakan ke enum uppercase
`attributes`	Magnitude, kedalaman, potensi tsunami, atribut peringatan	ID gunung, jumlah erupsi, tinggi abu, confidence jika ada

Tabel 4.1: Pemetaan kedua sumber. `hazard_id` dibuat saat ingest pertama; `ingested_at` mencatat waktu penerimaan.

Untuk BMKG tanpa peringatan tsunami yang berlaku, severity adalah NORMAL jika magnitude < 5, WASPADA jika $5 \leq m < 6{,}5$, dan SIAGA jika $m \geq 6{,}5$. Jika `potential_tsunami=true`, mapper mencari peringatan dengan `related_event_id` yang cocok dan mengambil tingkat ancaman tertinggi. Peringatan dari gempa lain tidak dipakai.

Mapper membaca objek JSON secara dinamis agar field tambahan tidak hilang hanya karena belum tercantum dalam struct aplikasi. Field yang diperlukan untuk pemetaan tetap divalidasi, sementara extension yang tidak dikenal dipertahankan dalam atribut, termasuk nilai nested. Angka extension juga dipertahankan tanpa terlebih dahulu mengubah seluruh nilai menjadi float.

Jika nama extension bertabrakan dengan atribut hasil pemetaan, nilai asli dari sumber ditempatkan dalam `source_extras`. Dengan cara ini, kedua nilai tetap dapat diperiksa. Sebaliknya, input yang membuat pemetaan ambigu atau tidak valid, seperti duplicate key, enum yang tidak dikenal, ID gunung di luar lookup, dan timestamp tanpa timezone, ditolak. Kebijakan ini menerima tambahan informasi sambil menjaga arti field canonical.

4.2. Perubahan skema saat sistem berjalan

PVMBG menyediakan endpoint `POST /admin/schema-version` yang dilindungi credential sumber. Pada versi 1, respons belum memiliki `confidence_level`; versi 2 menambahkan field tersebut. Karena polling mengambil snapshot penuh, Aggregator dapat melihat perubahan pada `report_id` yang sudah pernah disimpan.

Ketika isi record berubah, store mempertahankan `hazard_id`, menaikkan revisinya, lalu menyimpan snapshot baru di `hazard_versions`. Data terbaru dapat dibaca melalui API, sementara snapshot sebelumnya tetap tersedia melalui perintah audit milik Aggregator. Penyimpanan atribut dalam JSONB membuat penambahan confidence tidak memerlukan kolom atau migrasi baru.

Riwayat revisi dipertahankan agar bentuk sebelum dan sesudah perubahan dapat dibandingkan untuk hazard yang sama. Jika hanya record terbaru yang ditimpa, bentuk sebelumnya akan hilang. Konsekuensinya, storage bertambah pada setiap perubahan isi record.

Decoding ke struct dengan daftar field tetap sebenarnya lebih sederhana, tetapi field baru yang tidak dikenali dapat terbuang. Polling dengan cursor `since` juga dapat mengurangi transfer data, tetapi pada mock ini perubahan record lama dapat terlewat. Snapshot dipilih untuk memenuhi perubahan schema saat sistem berjalan. Konsekuensinya, jumlah data yang dipindahkan dan diperiksa pada setiap polling meningkat ketika sumber terus menghasilkan record.

4.3. Pengujian pemetaan dan perubahan schema

4.3.1 Membandingkan sumber dengan data canonical

Pengujian dimulai dengan membaca satu record BMKG menggunakan credential BMKG. `BMKG_REF` menyimpan `event_id` yang dipilih, kemudian filter `jq` menampilkan record dengan ID tersebut.

```
$ BMKG_REF=$(curl -sS -H "X-BMKG-Key: $BMKG_API_KEY" \
>   http://127.0.0.1:8281/seismic-events | jq -r '.[0].event_id')
> curl -sS -H "X-BMKG-Key: $BMKG_API_KEY" \
>   http://127.0.0.1:8281/seismic-events \
>   | jq --arg id "$BMKG_REF" '.[] | select(.event_id == $id)'
>
{
  "event_id": "sim-60f75b621ea2f87f-event-001",
  "magnitude": 4.2,
  "depth_km": 7,
  "epicenter_lat": -9.8,
  "epicenter_lon": 105.25,
  "region_name": "Wilayah Simulasi 01",
  "occurred_at": "2026-10-07T04:17:29.565895838Z",
  "potential_tsunami": false
}
```

Gambar 4.1: Request ke BMKG dan record sumber yang dipilih untuk dibandingkan.

Record yang sama kemudian dicari melalui API internal Aggregator. Request memakai header `X-Internal-Key` sebagai credential. Query `source=BMKG` membatasi sumber, sedangkan `view=full` meminta record canonical lengkap. Limit 100 digunakan agar record seed yang dipilih masih tercakup pada snapshot pengujian.

```
$ curl -sS -H "X-Internal-Key: $AGGREGATOR_API_KEY" \
>
'http://127.0.0.1:8283/internal/v1/hazards?source=BMKG&view=full&limit=100'
\
>   | jq --arg id "$BMKG_REF" '.data[] | select(.source_ref_id == $id)'
>
{
  "hazard_id": "c10e114a-2f04-400c-b3a0-110aaf9cc120",
  "source": "BMKG",
  "source_ref_id": "sim-60f75b621ea2f87f-event-001",
  "hazard_type": "SEISMIC",
  "severity": "NORMAL",
  "area_name": "Wilayah Simulasi 01",
  "latitude": -9.8,
  "longitude": 105.25,
  "occurred_at": "2026-10-07T04:17:29.565895+00:00",
  "ingested_at": "2026-10-07T04:55:41.058969+00:00",
  "attributes": {
    "depth_km": 7,
    "magnitude": 4.2,
    "potential_tsunami": false
  }
}
```

Gambar 4.2: Hasil pemetaan BMKG yang dibaca dari Canonical Store melalui Aggregator.

`source_ref_id` pada hasil sama dengan `event_id` sumber. Magnitude 4,2 tanpa potensi tsunami menghasilkan severity NORMAL, sedangkan region dan koordinat masuk ke field canonical. Magnitude, kedalaman, dan potensi tsunami tetap tersedia dalam atribut. Waktu kejadian cocok setelah presisi sumber dinormalisasi ke mikrodetik. Notasi `+00:00` pada hasil menyatakan UTC, sama seperti `Z` pada data sumber.

PVMBG diperiksa dengan urutan yang sama, menggunakan Bearer token sumber dan `report_id` sebagai identitas pembanding. Pada versi 1, record yang dipilih belum memiliki confidence.

```
$ PVMBG_REF=$(curl -sS -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports | jq -r '.[0].report_id')
> curl -sS -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports \
>   | jq --arg id "$PVMBG_REF" '.[] | select(.report_id == $id)'
>
{
  "report_id": "sim-19ac08b49fd13778-report-001",
  "volcano_id": "sim-tamriel-red-mountain",
  "alert_level": "Normal",
  "eruption_count_24h": 1,
  "ash_column_height_m": 125,
  "reported_at": "2026-10-07T03:58:29.494743062Z"
}
```

Gambar 4.3: Record PVMBG versi 1 sebelum penambahan field.

```
$ curl -sS -H "X-Internal-Key: $AGGREGATOR_API_KEY" \
>
'http://127.0.0.1:8283/internal/v1/hazards?source=PVMBG&view=full&limit=100'
\
>   | jq --arg id "$PVMBG_REF" '.data[] | select(.source_ref_id == $id)'
>
{
  "hazard_id": "a3e0c470-1b79-4f39-aeb1-c7a063568995",
  "source": "PVMBG",
  "source_ref_id": "sim-19ac08b49fd13778-report-001",
  "hazard_type": "VOLCANIC",
  "severity": "NORMAL",
  "area_name": "Gunung Simulasi Red Mountain (Tamriel)",
  "latitude": -7,
  "longitude": 108,
  "occurred_at": "2026-10-07T03:58:29.494743+00:00",
  "ingested_at": "2026-10-07T04:55:43.141503+00:00",
  "attributes": {
    "ash_column_height_m": 125,
    "eruption_count_24h": 1,
    "volcano_id": "sim-tamriel-red-mountain"
  }
}
```

Gambar 4.4: Hasil pemetaan record PVMBG yang sama ke HazardEvent.

Hasilnya menggunakan `VOLCANIC` dan severity NORMAL sesuai `alert_level=Normal`. ID gunung menunjuk fixture Red Mountain, sehingga lokasi canonical adalah $(-7, 108)$. Jumlah erupsi dan tinggi kolom abu tetap tersimpan dalam atribut. Perbandingan kedua sumber ini menunjukkan bahwa bentuk akhir konsisten meskipun nama field dan cara penentuan lokasi berbeda.

4.3.2 Menambahkan field ketika service tetap berjalan

Sebelum schema diubah, pengujian menyimpan identitas dan waktu mulai container serta membaca daftar migrasi. Request POST kemudian mengirim body `{"version":2}` ke kontrol mock PVMBG. Respons `version: 2` menunjukkan bahwa mock telah menerima perubahan; jeda 10 detik memberi kesempatan kepada polling berikutnya untuk mengambil schema baru.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'sort_by(.Name) | map({name: .Name, id: .Id,
>       started_at: .State.StartedAt, restarts: .RestartCount})' >
/tmp/report-session/schema-before.json
> docker compose exec -T aggregator /aggregator store-audit \
>   | jq '[.migrations[].version]'
> curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"version":2}' \
>   http://127.0.0.1:8282/admin/schema-version | jq .
> sleep 10
[
  "001_canonical_store.sql",
  "002_outbox_unpublished_index.sql"
]
{
  "version": 2
}
```

Gambar 4.5: Mencatat kondisi awal dan mengaktifkan schema PVMBG versi 2.

Perintah `history-audit` membaca revisi 1 dan 2 dari record yang sama melalui binary Aggregator. Opsi `-source-ref-id` menggunakan ID yang dipilih sebelum toggle. `tee` menyimpan output audit lengkap, sedangkan `jq` memilih identitas, atribut setiap revisi, dan daftar migrasi agar perubahannya mudah dibandingkan.

```
$ docker compose exec -T aggregator /aggregator history-audit \
>   --source PVMBG --source-ref-id "$PVMBG_REF" --revisions 1,2 \
>   | tee docs/report/evidence/schema-command-audit.json \
>   | jq '{hazard_id, source_ref_id,
>       versions: [.versions[] | {revision, attributes: .data.attributes}],
>       migrations: [.migrations[].version]}'
>
{
  "hazard_id": "a3e0c470-1b79-4f39-aeb1-c7a063568995",
  "source_ref_id": "sim-19ac08b49fd13778-report-001",
  "versions": [
    {
      "revision": 1,
      "attributes": {
        "volcano_id": "sim-tamriel-red-mountain",
        "eruption_count_24h": 1,
        "ash_column_height_m": 125
      }
    },
    {
      "revision": 2,
      "attributes": {
        "volcano_id": "sim-tamriel-red-mountain",
        "confidence_level": 0.58,
        "eruption_count_24h": 1,
        "ash_column_height_m": 125
      }
    }
  ],
  "migrations": [
    "001_canonical_store.sql",
    "002_outbox_unpublished_index.sql"
  ]
}
```

Gambar 4.6: Revisi 1 dan 2 memiliki identitas hazard yang sama; confidence muncul pada revisi 2.

Revisi 1 tidak memuat `confidence_level`, sedangkan revisi 2 memuat nilai 0,58. Kedua snapshot tetap dapat dibaca, dan daftar migrasi masih berisi dua migrasi awal. Jadi, penambahan field disimpan sebagai perubahan isi dokumen tanpa migrasi baru. Record sumber yang sudah dipetakan dan kedua revisi audit juga diperiksa dengan JSON Schema HazardEvent. Command dan hasil validasinya ditampilkan pada Gambar 4.8; validasi mencakup empat record trial 5 Oktober serta empat record yang ditangkap dari command 7 Oktober.

Pengujian terakhir membandingkan ID, waktu mulai, dan restart count seluruh container sebelum serta sesudah toggle. Output `Files ... are identical` berarti data inspeksi tersebut tidak berubah. Bersama hasil audit, ini menunjukkan bahwa schema baru diproses tanpa menjalankan ulang service.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'sort_by(.Name) | map({name: .Name, id: .Id,
>       started_at: .State.StartedAt, restarts: .RestartCount})' >
/tmp/report-session/schema-after.json
> diff -s /tmp/report-session/schema-before.json \
>   /tmp/report-session/schema-after.json
Files /tmp/report-session/schema-before.json and /tmp/report-session/schema-
after.json are identical
```

Gambar 4.7: Perbandingan kondisi container sebelum dan sesudah perubahan schema.

```
$ go run -mod=readonly docs/report/evidence/validate-records.go
PVMBG v1 mapping: HazardEvent schema valid
PVMBG revision 1: HazardEvent schema valid
PVMBG revision 2: HazardEvent schema valid
BMKG mapping: HazardEvent schema valid
7 Oktober, p1-bmkg-canonical: HazardEvent schema valid
7 Oktober, p1-pvmbg-canonical: HazardEvent schema valid
7 Oktober, PVMBG revision 1: HazardEvent schema valid
7 Oktober, PVMBG revision 2: HazardEvent schema valid
```

Gambar 4.8: Delapan record hasil pengujian lolos kontrak HazardEvent, termasuk revisi setelah perubahan schema.

4.3.3 Memeriksa field yang tidak dikenal

Penambahan confidence diuji melalui stack yang berjalan. Untuk variasi input lain, test mapper memeriksa penyimpanan field nested dan penanganan nama yang bertabrakan melalui `source_extras`. Test juga memastikan input yang tidak valid ditolak.

```
$ go test -mod=readonly -count=1 -v \
>   ./services/aggregator/internal/mapper \
>   -run 'TestMapVolcanicPreservesV1V2AndDynamicFields|TestMapperRejects'
=== RUN   TestMapVolcanicPreservesV1V2AndDynamicFields
--- PASS: TestMapVolcanicPreservesV1V2AndDynamicFields (0.00s)
=== RUN   TestMapperRejectsUnknownVolcanoInvalidEnumsAndDuplicateKeys
--- PASS: TestMapperRejectsUnknownVolcanoInvalidEnumsAndDuplicateKeys
(0.00s)
=== RUN   TestMapperRejectsTimestampWithoutTimezone
--- PASS: TestMapperRejectsTimestampWithoutTimezone (0.00s)
PASS
ok      github.com/ryonlunar/tugas-besar-
AAT/services/aggregator/internal/mapper    0.005s
```

Gambar 4.9: Test mapper untuk extension v1/v2 dan penolakan input yang tidak valid.

Kriteria	Bukti
P1.1	Record sumber dan canonical BMKG/PVMBG pada Gambar 4.1–4.4; trial lengkap di `integration.json`.
P1.2–3	Toggle, audit revisi, dan perbandingan container pada Gambar 4.5–4.7; validasi kontrak di `validated-records.log`.
P1.4	Kebijakan extension dan test pada Gambar 4.9.
P1.5	Mekanisme, alternatif, dan batas perubahan schema dijelaskan pada bab ini.

4.4. Batasan

Dukungan perubahan schema pada implementasi ini mencakup penambahan atribut dalam batas ukuran respons dan pesan. Jika field wajib berganti nama atau tipe, atau sumber memperkenalkan enum dengan makna baru, mapper perlu diperbarui. Sistem tidak dapat menyimpulkan sendiri bahwa sebuah field baru menggantikan field lama. Pengujian langsung membuktikan penambahan `confidence_level`, sedangkan variasi extension lain diperiksa melalui test mapper.

Pengambilan snapshot juga memiliki batas ketika volume data membesar. Pada pengujian terpisah dengan sekitar 1000 record PVMBG, perubahan schema menghasilkan `POLL_FAILED`, sehingga revisi baru belum tersimpan. Batas waktu write 2 detik untuk batch besar diduga berperan, tetapi penyebabnya belum dipastikan. Catatan pengujian tersedia pada `evidence/schema-volume-review.txt`.

Dengan demikian, keberhasilan pada profil seed M1 belum cukup untuk menyatakan bahwa perubahan schema akan selalu berhasil pada volume yang terus tumbuh. Sebelum skala dinaikkan, proses ingest perlu diukur ulang dan dipertimbangkan untuk menggunakan pagination, change feed, atau batch yang lebih kecil.

Problem 2: Concurrency, Availability, & Graceful Degradation

5.1. Isolasi request dan polling

Jalur request client dipisahkan dari pengambilan data sumber. Client API membaca data yang telah tersimpan melalui Aggregator, sehingga request tidak harus menunggu mock PVMBG yang dapat terlambat 3 detik. Sementara itu, loop polling BMKG dan PVMBG berjalan sendiri-sendiri; fetch seismic dan warning BMKG juga dapat dilakukan paralel. HTTP client menggunakan kembali koneksi untuk mengurangi biaya pembukaan koneksi pada request berikutnya.

Di database, pekerjaan baca memiliki pool 10 koneksi, sedangkan penulisan BMKG, penulisan PVMBG, dan relay masing-masing mendapat 2 koneksi. Dengan pemisahan ini, pekerjaan polling tidak mengambil seluruh koneksi yang dibutuhkan jalur baca. API dan Aggregator membatasi jumlah request yang sedang diproses menjadi 128. Jika batas tersebut tercapai, service mengembalikan 429 beserta `Retry-After`, sehingga client mengetahui bahwa request perlu dicoba lagi.

Timeout membatasi lamanya setiap pekerjaan: baca database 200 ms, write 2 detik, request API ke Aggregator 1 detik, request ke BMKG 1 detik, dan request ke PVMBG 4 detik. Angka PVMBG dibuat lebih panjang agar delay 3 detik pada pengujian masih dapat ditangani.

Alternatifnya adalah mengambil data sumber untuk setiap request client. Cara itu dapat memberikan data yang lebih baru, tetapi waktu respons dan kegagalan sumber langsung memengaruhi client. Polling menghindari ketergantungan tersebut dengan konsekuensi adanya jeda pembaruan. Karena itu, API menyertakan metadata freshness agar client mengetahui status sumber dan waktu pembaruan terakhir.

5.2. Outage dan pemulihan

Ketika polling PVMBG gagal, Aggregator mempertahankan data terakhir yang berhasil disimpan. API masih dapat mengembalikannya, tetapi `meta.sources.PVMBG.status` berubah menjadi `stale`. Respons juga menyertakan `last_success_at` dan `stale_since`, sehingga client mengetahui kapan sumber terakhir berhasil dibaca dan sejak kapan data dinyatakan tertinggal. Pada saat yang sama, BMKG dapat tetap berstatus `fresh` karena polling-nya terpisah.

Jika sumber belum pernah menghasilkan data yang dapat dilayani, API menyatakan `unavailable` dengan status 503. Setelah outage dinonaktifkan, polling berikutnya memperbarui data dan mengembalikan status PVMBG menjadi fresh tanpa restart API maupun Aggregator. Pengujian outage dilakukan melalui endpoint kontrol mock. Request berikut mengirim `enabled=true` ke `/admin/outage`; setelah respons diterima, pengujian menunggu polling mendeteksi kegagalan sumber.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"enabled":true}' \
>   http://127.0.0.1:8282/admin/outage | jq .
{
  "enabled": true
}

$ sleep 10
```

Gambar 5.1: Mengaktifkan outage pada sumber PVMBG.

Berikutnya, dua request dikirim ke Client API dengan token Tim Lapangan. Request pertama memilih PVMBG dan menampilkan jumlah record beserta metadata sumber, sedangkan request kedua membaca metadata BMKG. Pada hasil PVMBG masih ada lima record yang dikembalikan, tetapi statusnya stale dan waktu kegagalan tercatat. BMKG tetap fresh. API tetap menyediakan data terakhir beserta status dan waktu kegagalan sumber.

```
$ FIELD_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-session/field-
team.json)
> curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=PVMBG&view=full&limit=5' \
>   | jq '{records: (.data | length), source: .meta.sources.PVMBG}'
{
  "records": 5,
  "source": {
    "status": "stale",
    "last_success_at": "2026-10-07T05:15:29.209971Z",
    "stale_since": "2026-10-07T05:15:31.341025Z"
  }
}

$ curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1' \
>   | jq '.meta.sources.BMKG'
{
  "status": "fresh",
  "last_success_at": "2026-10-07T05:15:36.44186Z",
  "stale_since": null
}
```

Gambar 5.2: Respons API selama PVMBG outage: data terakhir tersedia dan kondisi setiap sumber dinyatakan secara eksplisit.

Pemulihan dilakukan dengan mengirim `enabled=false` ke endpoint yang sama. Setelah jeda polling, request API menunjukkan PVMBG kembali fresh dan `stale_since` menjadi

null. Langkah ini tidak menjalankan command restart. Trial lengkap pada 5 Oktober juga membandingkan kondisi container sebelum dan sesudah outage; hasilnya tersimpan di `outage-recovery.json`.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"enabled":false}' \
>   http://127.0.0.1:8282/admin/outage | jq .
{
  "enabled": false
}

$ sleep 10

$ curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=PVMBG&view=full&limit=1' \
>   | jq '.meta.sources.PVMBG'
{
  "status": "fresh",
  "last_success_at": "2026-10-07T05:15:47.835933Z",
  "stale_since": null
}
```

Gambar 5.3: Mematikan outage dan membaca hasil polling yang sudah pulih.

5.3. Metode dan hasil load test

Load test dijalankan pada 5 Oktober 2026 menggunakan `scripts/load/hazards-monotonic.py`. Script ini mengirim request HTTP dengan 30 worker BMKG dan 30 worker PVMBG secara bersamaan selama 90 detik. Tidak ada think time, sehingga setiap worker langsung mengirim request berikutnya setelah menerima respons.

Setiap worker mempunyai sesi Tim Lapangan sendiri, menggunakan HTTP keep-alive, dan membaca `view=full` dengan limit 50. Access token tetap menggunakan TTL 60 detik; script melakukan refresh sebelum expiry agar masa berlaku token tidak menjadi penyebab putusnya beban. Selama pengujian, delay sumber PVMBG ditetapkan 3000 ms.

Latency dihitung sejak request dimulai hingga seluruh body diterima, termasuk pembukaan koneksi jika diperlukan. Pengukuran memakai `time.perf_counter_ns`, yaitu timer monotonic yang tidak mengikuti perubahan wall clock host, sedangkan percentile dihitung dengan metode nearest rank.

Untuk memeriksa syarat koneksi paralel, collector menggunakan Linux `ss` dan mengambil sampel koneksi TCP established ke port API setiap sekitar 1 detik. Pengukuran ini diperlukan karena jumlah worker saja tidak membuktikan jumlah koneksi yang benar-benar terbuka.

Sebelum beban diberikan, delay mock diatur dengan body `min_ms` dan `max_ms` yang sama-sama bernilai 3000. Request langsung ke sumber kemudian mengukur waktu respons

melalui `curl`. Flag `-o /dev/null` membuang body, sedangkan `-w` menampilkan status HTTP dan durasi request. Ini membedakan delay sumber dari latency Client API yang membaca data tersimpan.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' \
>   --data '{"min_ms":3000,"max_ms":3000}' \
>   http://127.0.0.1:8282/admin/delay | jq .
> curl -sS -o /dev/null -w 'HTTP %{http_code}; durasi %{time_total} s\n' \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports
{
  "min_ms": 3000,
  "max_ms": 3000
}
HTTP 200; durasi 3.018729 s
```

Gambar 5.4: Mengatur delay 3 detik dan mengukur request langsung ke sumber PVMBG.

Collector TCP dijalankan bersamaan dengan load test untuk mengukur jumlah koneksi established selama beban berlangsung. Kedua proses dimulai dari root repo dengan command berikut:

```
export API_URL=http://127.0.0.1:8280
export AUTH_URL=http://127.0.0.1:8284
python3 scripts/load/capture-tcp-connections.py \
  --api-url "$API_URL" --seconds 110 --interval 1 \
  --output /tmp/report-tcp.json &
tcp_pid=$!
python3 scripts/load/hazards-monotonic.py \
  --seconds 90 --vus-per-source 30 --output /tmp/report-load.json
wait "$tcp_pid"
```

Hasil load test tersimpan di `docs/report/evidence/load.json`. Field yang diperiksa meliputi jumlah worker, durasi pengujian, p95 BMKG, throughput, dan persentase error.

```
$ jq '{started_at, profile, elapsed_seconds,
>     bmkg_p95_ms: .sources.BMKG.success_duration_ms.p95,
>     throughput: .hazard_successes_per_second,
>     error_percent: .hazard_unexpected_error_percent}' \
>   docs/report/evidence/load.json
{
  "started_at": "2026-10-05T13:11:41.596066+00:00",
  "profile": {
    "vus_per_source": 30,
    "duration_seconds": 90.0,
    "view": "full",
    "limit": 50,
    "identity": "field-team",
    "access_ttl_seconds": 60
  },
  "elapsed_seconds": 90.08667208399856,
  "bmkg_p95_ms": 124.521589,
  "throughput": 980.25599078252,
  "error_percent": 0.03396047001290498
}
```

Gambar 5.5: Membaca hasil load test 5 Oktober dari berkas pengukuran yang disimpan.

Metrik	BMKG	PVMBG
Request terkirim	44.510	43.828
Respons sukses	44.494	43.814
Error selain 429	16	14
HTTP 429 pada API	0	0
p50 sukses (ms)	52,07	52,64
p95 sukses (ms)	124,52	125,24
p99 sukses (ms)	190,71	194,30
p95 seluruh respons (ms)	124,76	125,44

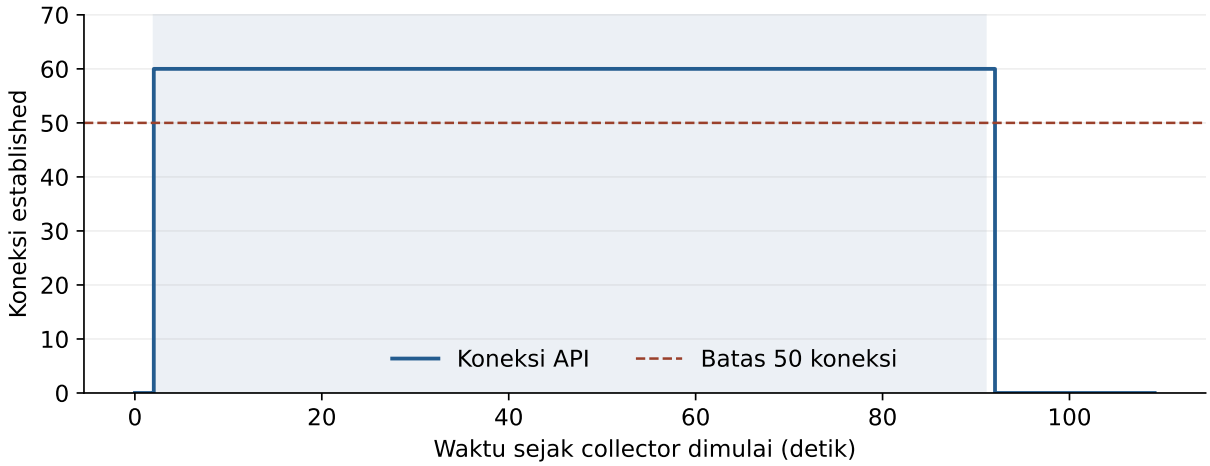
Tabel 5.1: Hasil beban simultan selama 90,09 detik. p50/p95/p99 sukses dihitung dari 2xx; p95 seluruh respons juga ditampilkan.

Dari 88.338 request yang dikirim, 88.308 mendapat respons sukses. Throughput-nya mencapai 980,26 respons sukses per detik, sedangkan 30 request lainnya menghasilkan HTTP 503, sebagaimana dicatat dalam `evidence/load-statuses.json`. Dengan demikian, error selain 429 sebesar 0,034%, masih di bawah batas 1%.

API tidak menghasilkan 429 pada pengujian ini. Auth menghasilkan 18 respons 429 selama login dan refresh, tetapi retry berhasil sehingga tidak ada sesi worker yang ga-

gal. Angka Auth tersebut dilaporkan terpisah agar pembatasan request terkendali tidak tercampur dengan error pada pembacaan hazard.

Delay langsung dari sumber PVMBG terukur 3.004,46 ms, sementara p95 respons sukses BMKG sebesar 124,52 ms. Hasil ini memenuhi batas 300 ms ketika kedua sumber dibaca bersamaan. Sampel TCP juga menunjukkan sedikitnya 50 koneksi pada interval 89,00 detik, melebihi syarat 60 detik. Karena collector mengambil sampel sekitar sekali per detik, kesimpulan mengenai koneksi mengikuti resolusi pengukuran tersebut.



Gambar 5.6: Sampel koneksi established selama pengujian. Garis putus-putus menunjukkan batas 50 koneksi. Interval di atas batas dihitung dari rangkaian sampel koneksi.

Kriteria	Bukti
P2.1	Delay sumber terukur dan BMKG p95 pada Tabel 5.1; kedua sumber menerima request bersamaan.
P2.2	Throughput, percentile, error, 429, serta sampel TCP; Tabel 5.1 dan Gambar 5.6.
P2.3	Kontrol outage, data stale, dan pemulihan pada Gambar 5.1–5.3; trial lengkap di `outage-recovery.json`.
P2.4	Metode, konfigurasi, serta mekanisme isolasi di bab ini; script dan output lengkap disertakan.

5.4. Batasan

Hasil load test menggambarkan beban yang dicoba pada satu host dengan pembacaan maksimal 50 record per request. Pengujian tersebut belum mencari kapasitas maksimum sistem. Selain itu, jalur baca tetap membutuhkan Aggregator dan PostgreSQL, sehingga isolasi dari gangguan sumber tidak menjamin API tersedia jika salah satu komponen ini mati.

Penilaian freshness masih menggunakan wall clock. Jika waktu host bergeser, umur data yang ditampilkan dapat ikut berubah meskipun pengukuran latency tetap benar. Perluasan ke beberapa node karena itu memerlukan sinkronisasi waktu serta pengujian ulang terhadap kapasitas dan pola kegagalan yang baru.

Problem 3: Service-to-Service Authentication & Trust

6.1. Credential dan hak akses

Setiap sumber memiliki credential sendiri: BMKG menerima API key melalui header `X-BMKG-Key`, sedangkan PVMBG menerima Bearer token sumber. API internal Aggregator memakai key lain. Pemisahan ini membuat kebocoran satu credential tidak otomatis memberikan akses ke seluruh antarmuka, dan credential sumber tidak dapat dipakai sebagai identitas client.

Untuk akses client, Auth membedakan identitas `media`, `field-team`, dan `bnpb-internal`. Media hanya memiliki scope ringkasan, yang berisi tepat tujuh field: `hazard_id`, `source`, `hazard_type`, `severity`, `area_name`, `occurred_at`, dan `ingested_at`. Scope lengkap menambahkan `source_ref_id`, koordinat, dan `attributes`.

Pembatasan dilakukan oleh Client API sebelum data diminta dari Aggregator. Jadi, Media tidak dapat memperoleh data raw hanya dengan mengubah query menjadi `view=full` atau meminta field tambahan secara eksplisit; kedua request tersebut ditolak dengan 403. Aturan akses berada di server, sehingga tidak bergantung pada tampilan yang dipakai client.

JWT ditandatangani Auth menggunakan RS256 dan private key. API hanya memerlukan public key untuk memeriksa signature, issuer, audience, expiry, serta scope. Verifikasi dilakukan secara lokal, sehingga token yang masih valid tetap dapat digunakan saat Auth berhenti sementara [12]. Jika API memakai shared signing secret, material untuk memverifikasi juga dapat digunakan untuk membuat token. Sementara itu, introspection ke Auth pada setiap request memudahkan pemeriksaan status token secara terpusat, tetapi menambahkan dependency pada jalur baca. Untuk M1, verifikasi lokal dipilih bersama TTL yang singkat.

6.2. Expiry dan refresh

Access token berlaku 60 detik. Agar client dapat melanjutkan sesi tanpa login ulang, Auth juga memberikan refresh token berupa nilai acak. Hanya hash token tersebut yang disimpan di SQLite. Setiap refresh menghasilkan token baru dan menandai token refresh sebelumnya sudah digunakan. Jika token lama dipakai kembali, keluarga sesi itu dibatalkan untuk mencegah kelanjutan refresh dari token yang digunakan ulang.

Batas umur keluarga sesi adalah 24 jam dan tetap dihitung dari pembentukan sesi, sehingga rotasi tidak memperpanjang sesi tanpa batas. Database Auth berada pada volume privat agar sesi refresh tidak hilang ketika container dijalankan ulang.

Rotasi refresh token tidak langsung mencabut access JWT yang belum expired. API tidak memiliki daftar revocation untuk access token; token tersebut masih dapat digunakan sampai `exp`. Pengujian menunggu expiry alami sebelum memeriksa penolakan token lama dengan status 401.

6.3. Pengujian credential dan hak akses

6.3.1 Credential hanya berlaku pada sumbernya

Pengujian credential dilakukan sebagai empat request terpisah. Untuk BMKG, header tetap bernama `X-BMKG-Key`: request pertama mengisinya dengan key BMKG, sedangkan request kedua memakai token PVMBG sebagai nilai yang salah. Status HTTP setiap request dibaca melalui opsi `-w`.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "X-BMKG-Key: $BMKG_API_KEY" \
>   http://127.0.0.1:8281/seismic-events
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "X-BMKG-Key: $PVMBG_TOKEN" \
>   http://127.0.0.1:8281/seismic-events
HTTP 401
```

Gambar 6.1: BMKG menerima key miliknya dengan 200 dan menolak credential PVMBG dengan 401.

Arah sebaliknya diuji pada PVMBG dengan skema Bearer yang sama untuk kedua request. Token PVMBG menghasilkan 200, sedangkan key BMKG menghasilkan 401. Karena format header tetap sesuai dengan antarmuka sumber, status 401 menunjukkan bahwa sumber menolak nilai credential yang salah.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $BMKG_API_KEY" \
>   http://127.0.0.1:8282/volcanic-reports
HTTP 401
```

Gambar 6.2: PVMBG menerima token miliknya dan menolak key BMKG.

6.3.2 Media menerima ringkasan dan tidak dapat meminta data raw

Media terlebih dahulu login ke Auth dengan identitas dan secret client-nya sendiri. Respons login disimpan pada file privat untuk digunakan pada request berikutnya. Scope `hazards:summary` menjadi dasar pembatasan di API.

```
$ umask 077
> curl -sS --fail-with-body \
>   -H 'Content-Type: application/json' \
>   --data
"{\"client_id\":\"media\",\"client_secret\":\"$MEDIA_CLIENT_SECRET\"}" \
>   http://127.0.0.1:8284/v1/auth/login -o .secrets/report-
session/media.json
> jq '{token_type, expires_in, scope}' .secrets/report-session/media.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary"
}
```

Gambar 6.3: Auth memberikan scope ringkasan dan TTL 60 detik untuk Media.

Access token dari respons login dipasang pada header Authorization ketika Media membaca satu record BMKG. Filter `keys` menampilkan daftar field yang diterima: jumlahnya tepat tujuh dan tidak mencakup koordinat, `source_ref_id`, maupun `attributes`.

```
$ MEDIA_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-
session/media.json)
> curl -sS -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&limit=1' \
>   | jq '.data[0] | keys'
[
  "area_name",
  "hazard_id",
  "hazard_type",
  "ingested_at",
  "occurred_at",
  "severity",
  "source"
]
```

Gambar 6.4: Field respons Media sesuai dengan kontrak ringkasan.

Media kemudian mencoba `view=full` dan `fields=attributes` melalui dua request tersendiri. Keduanya ditolak dengan 403. Hasil ini menunjukkan bahwa query yang diubah client tidak dapat melewati pembatasan server.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?view=full'
HTTP 403

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?fields=attributes'
HTTP 403
```

Gambar 6.5: Dua cara meminta data raw sama-sama ditolak untuk token Media.

6.4. Pengujian expiry dan refresh

Login Tim Lapangan menghasilkan scope `hazards:summary hazards:raw` dan access token berumur 60 detik. Access token dari respons login digunakan pada request API berikutnya.

```
$ umask 077
> curl -sS --fail-with-body \
>   -H 'Content-Type: application/json' \
>   --data "{\"client_id\":\"field-
team\",\"client_secret\":\"$FIELD_TEAM_CLIENT_SECRET\"}" \
>   http://127.0.0.1:8284/v1/auth/login -o .secrets/report-session/field-
team.json
> jq '{token_type, expires_in, scope}' .secrets/report-session/field-
team.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary hazards:raw"
}
```

Gambar 6.6: Login Tim Lapangan memberikan izin ringkasan dan data raw.

Command berikut membaca payload JWT untuk menampilkan `iat` dan `exp`, kemudian menghitung selisihnya. Decoding pada terminal hanya digunakan untuk inspeksi claim; verifikasi token tetap dilakukan API ketika request dikirim. Selisihnya 60 detik dan token yang belum expired memperoleh 200.

```
$ FIELD_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-session/field-
team.json)
> jq -r .access_token .secrets/report-session/field-team.json \
>   | jq -R 'split(".")[1] | @base64d | fromjson | {iat, exp, ttl: (.exp-
.iat)}'
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
{
  "iat": 1791349941,
  "exp": 1791350001,
  "ttl": 60
}
HTTP 200
```

Gambar 6.7: Memeriksa TTL dari claim dan menggunakan access token sebelum expiry.

Selanjutnya, loop menunggu wall clock melewati `exp` dari token yang sama. Request API setelah batas tersebut menghasilkan 401.

```
$ ACCESS_EXP=$(jq -r .access_token .secrets/report-session/field-team.json \
>   | jq -R 'split(".")[1] | @base64d | fromjson | .exp')
> while [ "$(date +%s)" -le "$ACCESS_EXP" ]; do sleep 1; done
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 401
```

Gambar 6.8: Token lama ditolak setelah waktu expiry terlewati.

Refresh dilakukan dengan mengambil `refresh_token` dari file login dan mengirimkannya sebagai body JSON ke Auth. Opsi `-data-binary @-` mengirim body JSON dari pipeline, yang berisi refresh token dari respons login. Respons baru disimpan pada file lain dan menunjukkan TTL 60 detik serta scope yang sama. Tidak ada request login pada langkah ini.

```
$ jq '{refresh_token}' .secrets/report-session/field-team.json \
>   | curl -sS --fail-with-body -H 'Content-Type: application/json' \
>       --data-binary @- http://127.0.0.1:8284/v1/auth/refresh \
>       -o .secrets/report-session/refreshed.json
> jq '{token_type, expires_in, scope}' .secrets/report-
session/refreshed.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary hazards:raw"
}
```

Gambar 6.9: Mendapatkan token baru melalui refresh tanpa login ulang.

Token baru kemudian mendapat 200 ketika digunakan ke API, sementara token lama tetap mendapat 401. Trial integrasi 5 Oktober juga memeriksa sesi refresh setelah restart Auth, replay refresh token, dan penggunaan JWT yang masih valid ketika Auth berhenti; bukti tambahannya ada di `integration.json`.

```
$ NEW_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-
session/refreshed.json)
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $NEW_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 401
```

Gambar 6.10: Token baru dapat digunakan, sedangkan access token lama tetap expired.

Kriteria	Bukti
P3.1	Empat request pada Gambar 6.1 dan 6.2.
P3.2	Tujuh field Media dan penolakan 403 pada Gambar 6.4 serta 6.5.
P3.3	TTL, expiry alami, refresh, serta pemeriksaan token lama dan baru pada Gambar 6.7–6.10.
P3.4	Secret pada `.env.example` kosong; `.env` dan `.secrets/` diabaikan Git serta dikecualikan dari build context. Key dan sesi tersimpan pada volume privat.
P3.5	Pilihan autentikasi, trade-off, dan batas keamanan dijelaskan pada bab ini.

6.5. Menelusuri request dengan correlation ID

Request dapat diberi header `X-Correlation-ID` agar jejaknya dapat dicari di log API dan Aggregator. Pada contoh berikut, ID ditentukan oleh client dan diteruskan ke Aggregator. ID ini menghubungkan catatan dari request yang sama pada kedua service.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $NEW_ACCESS_TOKEN" \
>   -H 'X-Correlation-ID: report-command-correlation' \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
>
HTTP 200
```

Gambar 6.11: Request client dengan correlation ID yang ditentukan secara eksplisit.

Command log kemudian memilih catatan dengan ID tersebut. Keluaran menunjukkan operasi API, pembacaan upstream, dan operasi Aggregator yang semuanya memiliki correlation ID sama.

```
$ docker compose logs --no-color --no-log-prefix --since 2m \
>   client-api aggregator \
>   | jq -R 'fromjson? | select(.correlation_id == "report-command-
correlation")
>       | {service, correlation_id, operation, status, duration_ms}'
>
{
  "service": "aggregator",
  "correlation_id": "report-command-correlation",
  "operation": "GET /internal/v1/hazards",
  "status": 200,
  "duration_ms": 2
}
{
  "service": "client-api",
  "correlation_id": "report-command-correlation",
  "operation": "aggregator_read",
  "status": 200,
  "duration_ms": 3
}
{
  "service": "client-api",
  "correlation_id": "report-command-correlation",
  "operation": "GET /v1/hazards",
  "status": 200,
  "duration_ms": 4
}
```

Gambar 6.12: Log kedua service dapat ditelusuri melalui correlation ID yang sama.

6.6. Batasan keamanan dan waktu

Expiry JWT mengikuti wall clock. Pada implementasi ini, `iat` yang tampak berada di masa depan ketika clock mundur tidak dijadikan alasan tunggal untuk menolak token. API tetap memeriksa signature, issuer, audience, scope, expiry, serta `nbf` jika claim tersebut ada. Selisih `exp-iat` tetap 60 detik, tetapi pergeseran waktu host dapat mengubah kapan batas itu tercapai: clock mundur dapat memperpanjang masa berlaku yang dirasakan, sedangkan clock maju dapat membuat token lebih cepat expired. Sinkronisasi waktu diperlukan agar TTL sesuai dengan waktu nyata.

Di luar masalah waktu, komunikasi HTTP internal belum memakai TLS dan domain event masih menggunakan token NATS bersama. Sistem juga belum menyediakan rotasi key RSA maupun pencabutan access JWT secara langsung. Sebelum digunakan pada jaringan yang tidak dipercaya, perlindungan koneksi dan pemisahan credential broker perlu ditambahkan.

Problem 4: Arsitektur Microservice & Flexible Storage

7.1. Container dan ownership

Stack utama terdiri dari tujuh container aplikasi serta dua container infrastruktur, yaitu PostgreSQL dan NATS. Setiap aplikasi memiliki Dockerfile sendiri, sementara kedua komponen infrastruktur memakai image resmi. Third Consumer hanya dijalankan melalui profile demo. Compose membantu mengatur jaringan dan urutan startup, tetapi satu aplikasi tetap dapat dibangun dan diganti dengan `-no-deps` tanpa menjalankan ulang seluruh stack.

Canonical Store menjadi tanggung jawab Aggregator. Hanya Aggregator dan PostgreSQL yang terhubung ke `store-net`, dan PostgreSQL tidak membuka host port. Karena itu, Client API membaca data melalui HTTP Aggregator. Consumer memperoleh data melalui NATS, lalu menyimpan hasil olahan dalam database lokalnya sendiri. Perintah audit juga dijalankan melalui binary Aggregator agar inspeksi data tetap dilakukan oleh service pemiliknya.

Pembagian tersebut menghindari ketergantungan consumer pada schema tabel internal Aggregator. Sebagai konsekuensinya, tampilan consumer diperbarui secara eventual: perubahan canonical harus melewati outbox dan broker sebelum diterapkan pada database consumer. Shared database antarservice dapat mengurangi jumlah antarmuka HTTP, tetapi perubahan tabel akan mengikat beberapa aplikasi sekaligus dan tidak memenuhi ownership storage yang diminta spesifikasi.

7.2. Struktur storage dan alternatif

Field canonical yang stabil disimpan dalam kolom bertipe dengan constraint, sedangkan `attributes` memakai JSONB untuk menampung atribut yang dapat bertambah. Data terbaru berada di `hazard_events` dan setiap revisinya dicatat dalam `hazard_versions`. Dua tabel lain melengkapi alur ingest: `source_state` menyimpan kondisi polling, sementara `outbox` mencatat pesan yang perlu dipublikasikan.

Dengan struktur ini, tambahan `confidence_level` masuk ke dokumen atribut tanpa `ALTER TABLE`. Pilihan tersebut dibandingkan dengan dua alternatif pada tabel berikut.

7.3. Pengujian penggantian satu container

Pengujian menyimpan ID, waktu mulai, dan restart count container selain Dashboard sebelum melakukan rebuild. Dashboard dikecualikan dari daftar ini karena container itulah yang akan diganti.

Pilihan	Kelebihan	Konsekuensi
PostgreSQL + JSONB (dipilih)	Constraint identitas, query canonical, dan transaksi outbox dalam satu store	Atribut dinamis tetap perlu validasi; query JSONB yang sering dipakai dapat membutuhkan index tambahan.
Relasional dengan kolom tetap	Tipe setiap atribut jelas dan mudah di-query	Field baru membutuhkan migrasi; kurang sesuai perubahan schema langsung pada P4.
Document store, misalnya MongoDB	Bentuk dokumen sumber mudah disimpan dan berubah	Aturan canonical, index, dan transaksi history/outbox tetap perlu dirancang; fleksibilitas dokumen saja tidak menjamin konsistensi.

Tabel 7.1: Perbandingan pilihan penyimpanan. Fleksibilitas dokumen tidak menggantikan validasi kontrak.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'map(select(.Config.Labels["com.docker.compose.service"]
>       != "dashboard-consumer")) | sort_by(.Name)
>       | map({name: .Name, id: .Id, started_at: .State.StartedAt,
>           restarts: .RestartCount})' > .secrets/report-session/rebuild-
before.json
```

Gambar 7.1: Mencatat kondisi container lain sebelum Dashboard dibangun ulang.

Dashboard kemudian dihentikan dan image-nya dibangun kembali. Log build disimpan ke file, lalu `tail -n 6` membaca bagian akhir yang menunjukkan image berhasil dibangun. Log lengkap tersedia pada `evidence/dashboard-rebuild.log`.

```
$ docker compose stop dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Stopping
 Container aat-report-revision-dashboard-consumer-1  Stopped

$ docker compose build dashboard-consumer \
>   > .secrets/report-session/rebuild.log 2>&1
> tail -n 6 .secrets/report-session/rebuild.log

#15 [dashboard-consumer] exporting to image
#15 exporting layers done
#15 writing image
sha256:c54f9851d0aa3b832644178aac40303d6fd0d5e90ffaaecaa5989e5071680245 done
#15 naming to docker.io/library/aat-report-revision-dashboard-consumer done
#15 DONE 0.0s
```

Gambar 7.2: Menghentikan Dashboard dan membangun image service tersebut.

Saat Dashboard masih berhenti, request BMKG ke API menghasilkan 200. Dashboard kemudian dijalankan kembali dengan `-no-deps -force-recreate`; opsi pertama menjaga dependency tidak ikut dijalankan ulang, sedangkan opsi kedua memastikan container Dashboard diganti. `-wait` menunggu hasil healthcheck-nya siap.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 200

$ docker compose up -d --no-deps --force-recreate --wait dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Recreate
 Container aat-report-revision-dashboard-consumer-1  Recreated
 Container aat-report-revision-dashboard-consumer-1  Starting
 Container aat-report-revision-dashboard-consumer-1  Started
 Container aat-report-revision-dashboard-consumer-1  Waiting
 Container aat-report-revision-dashboard-consumer-1  Healthy
```

Gambar 7.3: API tetap merespons saat Dashboard mati, lalu Dashboard diganti secara terpisah.

Inspeksi diulang dengan proyeksi yang sama, kemudian `diff -s` membandingkan file sebelum dan sesudah. Keduanya identik dan berisi delapan container lain. Jadi, penggantian Dashboard tidak mengubah ID, waktu mulai, maupun restart count service lainnya. Trial integrasi 5 Oktober juga mencatat 10 request API sukses selama rebuild; rincian pengujian tersebut tetap tersedia di `integration.json`.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'map(select(.Config.Labels["com.docker.compose.service"]
>       != "dashboard-consumer")) | sort_by(.Name)
>       | map({name: .Name, id: .Id, started_at: .State.StartedAt,
>           restarts: .RestartCount})' > .secrets/report-session/rebuild-
after.json
> diff -s .secrets/report-session/rebuild-before.json \
>   .secrets/report-session/rebuild-after.json
> jq length .secrets/report-session/rebuild-after.json
Files .secrets/report-session/rebuild-before.json and .secrets/report-
session/rebuild-after.json are identical
8
```

Gambar 7.4: Delapan container lain tidak berubah setelah Dashboard diganti.

7.4. Pengujian ownership dan flexible storage

Inspeksi jaringan menunjukkan bahwa `store-net` hanya memuat Aggregator dan PostgreSQL. Inspeksi port PostgreSQL juga menunjukkan bahwa port database tidak dipublikasikan ke host. Hasil ini mendukung batas akses pada konfigurasi; kode Client API

dan consumer tetap diperiksa untuk memastikan keduanya tidak membuka koneksi SQL ke Canonical Store.

```
$ docker network inspect "${COMPOSE_PROJECT_NAME}_store-net" \
>   | jq '.[0].Containers | [.[] | .Name] | sort'
> docker inspect $(docker compose ps -q postgres) \
>   --format '{{json .NetworkSettings.Ports}}'
>
[
  "aat-report-revision-aggregator-1",
  "aat-report-revision-postgres-1"
]
{}
```

Gambar 7.5: Anggota jaringan storage dan port PostgreSQL yang tidak dipublikasikan.

Fleksibilitas storage dibuktikan bersama pengujian P1 pada Gambar 4.6. Snapshot revisi 1 dan 2 tetap terbaca dengan daftar migrasi yang sama. Pembacaan tersebut dilakukan melalui `/aggregator history-audit`, sehingga perubahan dokumen diperiksa oleh service pemilik storage.

Kriteria	Bukti
P4.1	Rebuild, request API, dan perbandingan delapan container pada Gambar 7.1–7.4.
P4.2	Revisi v1/v2 dan daftar migrasi pada Gambar 4.6.
P4.3	Anggota `store-net`, port privat, serta jalur baca pada kode; Gambar 7.5.
P4.4	Batas service pada Bab 2 dan perbandingan tiga pilihan storage pada bab ini.

7.5. Batasan

Batas ownership diterapkan pada aplikasi dan jaringan Compose. Administrator Docker tetap dapat memasuki container atau memasang volume, sehingga pemisahan ini tidak membatasi administrator host. Pengujian rebuild juga hanya membuktikan isolasi satu consumer; ketersediaan API saat Aggregator atau PostgreSQL mati belum dibuktikan oleh pengujian tersebut.

Selain itu, history, inbox, dan outbox yang telah dipublikasikan belum memiliki kebijakan pembersihan jangka panjang. Kebutuhan ruang akan terus meningkat jika data dibiarkan bertambah. Masalah batch snapshot besar yang dibahas pada P1 serta kebijakan retention lokal perlu diselesaikan sebelum sistem digunakan untuk volume yang lebih tinggi.

Problem 5: Fan-out Event Distribution/Pub-sub

8.1. Producer, stream, dan consumer

Distribusi event dimulai dari transaksi penyimpanan Aggregator. Event canonical, revisinya, dan pesan outbox ditulis bersamaan, sehingga event yang berhasil disimpan mempunyai catatan untuk dikirim ke broker. Relay kemudian mempublikasikan envelope ke subject `hazard.events.v1.BMKG` atau `hazard.events.v1.PVMBG`. Kedua subject ditangkap oleh stream `HAZARD_EVENTS` melalui filter `hazard.events.v1.*`.

Outbox dipilih untuk menangani celah antara penyimpanan data dan publish. Jika publish langsung dilakukan setelah commit, proses dapat mati ketika data sudah tersimpan tetapi pesan belum terkirim. Sebaliknya, publish sebelum commit dapat mengirim event dari transaksi yang kemudian gagal. Outbox mencatat kebutuhan publish dalam transaksi data yang sama, lalu relay mencobanya kembali. Konsekuensinya, ada jeda sebelum event sampai ke consumer dan kemungkinan publish berulang.

Producer mempublikasikan pesan berdasarkan subject dan kontrak event, sehingga penambahan pembaca tidak memerlukan perubahan pada producer. Daftar consumer serta posisi baca masing-masing dikelola oleh broker.

Dashboard dan Notifier memakai durable yang berbeda, yaitu `dashboard-v1` dan `notifier-v1`. Broker menyimpan posisi baca setiap durable meskipun aplikasinya berhenti. Dengan posisi yang terpisah, masing-masing aplikasi dapat menerima seluruh event dan memberikan ACK sendiri.

Jika kedua aplikasi berbagi satu durable, pesan akan dibagi sebagai pekerjaan untuk pembaca durable tersebut. Pola itu tidak memenuhi kebutuhan fan-out, yaitu event yang sama dapat diterima oleh setiap aplikasi. Karena itu, kedua consumer memakai pull consumer yang independen meskipun stream-nya sama.

Pull dipilih agar consumer meminta batch berikutnya setelah pengolahan batch sebelumnya selesai. Dibanding push, laju pengambilan lebih langsung dikendalikan aplikasi; ukuran batch dan batas ACK membatasi pesan yang sedang ditangani. Saat stream kosong, consumer tetap melakukan fetch dengan timeout.

Konfigurasi	Nilai dan fungsi
Storage / replica / retention	File / 1 / LimitsPolicy; ACK consumer tidak menghapus event dari stream.
Batas retention	24 jam, 256 MiB, 100.000 pesan; batas yang tercapai lebih dahulu menentukan kapasitas.
Batas satu pesan	64 KiB, termasuk ukuran yang diperhitungkan publisher.
Saat kapasitas penuh	DiscardNew; publish baru ditolak, bukan menghapus pesan lama hanya untuk menerima pesan baru.
Dedup broker	Message ID `hazard_id:revision`; window 2 menit.
ACK / redelivery	AckExplicit, AckWait 30 detik, MaxDeliver -1; retry tidak dibatasi jumlahnya.
Pull / inflight	Batch 10, expiry fetch 2 detik, MaxAckPending 64.

Tabel 8.1: Konfigurasi JetStream dan consumer yang digunakan pada M1.

LimitsPolicy mempertahankan event sampai batas retention, walaupun consumer telah memberikan ACK. Dengan DeliverAll, consumer baru dapat membaca event lama yang masih tersimpan. InterestPolicy juga mendukung fan-out, tetapi menghapus pesan setelah seluruh consumer yang berkepentingan memberikan ACK. Sementara itu, WorkQueuePolicy ditujukan untuk pembagian pekerjaan dengan satu pemroses per pesan [5]. LimitsPolicy dipilih agar penambahan consumer pada P5 dapat disertai replay, dengan biaya penyimpanan event selama masa retention.

Alternatif producer memanggil setiap consumer melalui HTTP membutuhkan daftar tujuan dan retry per tujuan di producer. NATS Core memberi fan-out sederhana, tetapi tidak menyimpan backlog durable untuk consumer yang mati. JetStream dipilih agar replay dan posisi baca dikelola broker. Kafka atau RabbitMQ juga memungkinkan pola sejenis, tetapi kebutuhan M1 dapat dipenuhi dengan JetStream dan konfigurasi lokal yang lebih kecil.

8.2. Semantik pengiriman dan idempotensi

Semantik pengiriman sistem adalah **at-least-once**, sehingga sebuah pesan dapat diterima lebih dari sekali. Misalnya, broker sudah menerima publish, tetapi relay gagal mencatat status published di database. Relay akan mencoba kembali. Hal serupa dapat terjadi ketika consumer sudah melakukan commit lokal, tetapi proses mati sebelum ACK diterima broker. Message ID `hazard_id:revision` membantu broker mengenali publish ulang selama dedup window 2 menit, tetapi perlindungan tersebut memiliki batas waktu.

Untuk menangani pengiriman ulang, setiap consumer menyimpan `message_id` pada inbox dengan constraint unik. Pencatatan inbox dan hasil olahan dilakukan dalam satu transaksi SQLite, kemudian consumer mengirim ACK. Jika pesan yang sama datang kembali, inbox menunjukkan bahwa hasilnya sudah diterapkan sehingga side effect lokal tidak diulang. Dashboard hanya menerima revisi yang lebih baru, sedangkan Notifier menggunakan identitas pesan yang unik untuk alert-nya.

Pesan dengan envelope tidak valid disimpan ke quarantine sebelum ACK, agar pesan rusak tidak terus diulang dan payload tetap tersedia untuk diperiksa. Jika penyimpanan gagal, consumer meminta pengiriman ulang sehingga pesan belum dianggap selesai.

Dengan demikian, hasil lokal dibuat idempoten meskipun delivery dapat berulang. Mekanisme ini tidak menyatakan bahwa seluruh alur bersifat exactly-once, terutama jika nantinya Notifier melakukan panggilan ke layanan SMS eksternal yang berada di luar transaksi SQLite.

Saat publish gagal, relay mencoba kembali dengan backoff 1 sampai 30 detik. Pending outbox dibatasi 10.000 pesan; jika batas ini tercapai, ingest tidak terus menambah antrean. Penghentian satu consumer karena itu tidak langsung menghalangi producer, selama broker masih dapat menyimpan event. Namun, kapasitas broker dan disk tetap menjadi batas: dengan DiscardNew, publish baru ditolak ketika stream penuh dan pesan tetap menunggu di outbox.

8.3. Pengujian fan-out dan recovery

8.3.1 Mencatat posisi dua durable consumer

Pengujian membaca endpoint monitoring JetStream melalui `wget` di dalam container NATS. Port monitoring tidak dibuka ke host. Query `streams=true&consumers=true` meminta informasi stream beserta consumer-nya, kemudian `jq` memilih dua durable utama.

Pada output, `last_seq` adalah sequence terakhir stream, sedangkan `ack` menampilkan ACK floor consumer dalam sequence stream. ACK floor menunjukkan posisi yang sudah diakui secara berurutan. `pending` menghitung pesan yang belum dikirim, dan `inflight` menghitung pesan yang sudah dikirim tetapi belum di-ACK. Karena dua jenis antrean ini berbeda, pending nol saja belum membuktikan recovery selesai.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 308,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 308,
      "pending": 0,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 308,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.1: Posisi awal stream dan dua durable utama sebelum Dashboard dihentikan.

Pada snapshot awal, Dashboard dan Notifier sama-sama memiliki ACK 308, sesuai sequence terakhir stream. Ini menunjukkan keduanya membaca stream yang sama dengan posisi masing-masing.

8.3.2 Menghentikan satu consumer dan membaca backlog

Dashboard dihentikan melalui Compose, sementara producer dan Notifier dibiarkan berjalan. Jeda 15 detik memberi kesempatan bagi mock untuk menghasilkan event baru dan Aggregator untuk meneruskannya ke broker.

```
$ docker compose stop dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Stopping
 Container aat-report-revision-dashboard-consumer-1  Stopped

$ sleep 15
```

Gambar 8.2: Menghentikan Dashboard tanpa menghentikan producer, broker, atau Notifier.

Command monitoring yang sama kemudian dijalankan ulang. Sequence stream naik dari 308 menjadi 311 dan ACK Notifier ikut mencapai 311. ACK Dashboard tetap 308 dengan tiga pesan pending. Jadi, event baru tetap tersimpan dan diolah Notifier saat Dashboard tidak aktif. Sequence 311 dicatat sebagai target yang harus dicapai Dashboard setelah kembali berjalan.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                 pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 311,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 308,
      "pending": 3,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 311,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.3: Stream dan Notifier terus maju, sedangkan backlog Dashboard bertambah.

Dashboard dijalankan kembali, kemudian pengujian menunggu 35 detik sebelum membaca posisi ACK. Jeda ini juga memberi ruang untuk redelivery jika masih ada pesan yang dikirim sebelum consumer berhenti; konfigurasi AckWait adalah 30 detik.

```
$ docker compose start dashboard-consumer
 Container aat-report-revision-nats-1  Waiting
 Container aat-report-revision-nats-1  Healthy
 Container aat-report-revision-dashboard-consumer-1  Starting
 Container aat-report-revision-dashboard-consumer-1  Started

$ sleep 35
```

Gambar 8.4: Menjalankan kembali Dashboard dan menunggu pemrosesan backlog.

Pada pemeriksaan berikutnya, ACK Dashboard mencapai 318, melewati target 311 yang dicatat saat ia mati. Notifier juga mencapai 318, dan kedua consumer tidak mempunyai pesan pending maupun inflight pada snapshot tersebut. Hasil ini membuktikan Dashboard memproses event yang terbit selama gangguan, bukan hanya menerima event setelah startup.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                 pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 318,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 318,
      "pending": 0,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 318,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.5: Dashboard sudah mengakui seluruh pesan hingga melewati target periode gangguan.

Inspeksi database lokal kemudian menunjukkan bahwa kedua consumer masing-masing memiliki 319 pesan pada inbox dan quarantine nol. Pemeriksaan ini dilakukan setelah snapshot broker, sehingga jumlah inbox dapat bertambah ketika event baru masuk. Inbox yang terpisah mendukung bahwa setiap aplikasi mengolah pesan secara independen.

```
$ docker compose exec -T dashboard-consumer /dashboard-consumer inspect \
>   | jq '{inbox_count, quarantine_count}'
{
  "inbox_count": 319,
  "quarantine_count": 0
}

$ docker compose exec -T notifier-consumer /notifier-consumer inspect \
>   | jq '{inbox_count, quarantine_count}'
{
  "inbox_count": 319,
  "quarantine_count": 0
}
```

Gambar 8.6: Hasil olahan berada pada inbox lokal masing-masing consumer.

8.3.3 Menambahkan consumer tanpa mengubah producer

Penambahan consumer diuji pada sesi terpisah. Sebelum Third Consumer dijalankan, ID, waktu mulai, dan restart count Aggregator disimpan. Profile `demo` kemudian menambahkan container Third Consumer dengan durable `third-consumer-v1`, tanpa membangun atau menjalankan ulang Aggregator. Consumer ini memakai binary Notifier dengan durable dan volume miliknya sendiri.

```
$ docker inspect $(docker compose ps -q aggregator) \
>   | jq -S '.[0] | {id: .Id, started_at: .State.StartedAt,
>       restarts: .RestartCount}' > /tmp/report-session/producer-before.json
> docker compose --profile demo up -d --no-deps --wait third-consumer
> sleep 5
#0 building with "default" instance using docker driver

#1 [third-consumer internal] load build definition from Dockerfile
#1 transferring dockerfile: 694B done
#1 DONE 0.1s

#2 [third-consumer internal] load metadata for
docker.io/library/golang:1.23-alpine
#2 DONE 0.0s

#3 [third-consumer internal] load .dockerignore
#3 transferring context: 676B done
#3 DONE 0.0s

#4 [third-consumer build 1/7] FROM docker.io/library/golang:1.23-alpine
#4 DONE 0.0s

#5 [third-consumer internal] load build context
#5 transferring context: 11.83kB 0.0s done
#5 DONE 0.0s

#6 [third-consumer build 4/7] COPY contracts ./contracts
#6 CACHED

#7 [third-consumer build 7/7] RUN go mod download && CGO_ENABLED=0
GOOS=linux go build -trimpath -ldflags="-s -w" -o /out/notifier-consumer
./services/notifier-consumer/cmd/server && mkdir /out/data
#7 CACHED

#8 [third-consumer stage-1 1/3] COPY --from=build /out/notifier-consumer
/notifier-consumer
#8 CACHED

#9 [third-consumer stage-1 2/3] COPY --from=build /etc/ssl/certs/ca-
certificates.crt /etc/ssl/certs/ca-certificates.crt
#9 CACHED

#10 [third-consumer build 3/7] COPY go.mod go.sum ./
#10 CACHED

#11 [third-consumer build 6/7] COPY services/notifier-consumer
./services/notifier-consumer
#11 CACHED

#12 [third-consumer build 2/7] WORKDIR /src
#12 CACHED

#13 [third-consumer build 5/7] COPY pkg/jetstreamconsumer
./pkg/jetstreamconsumer
#13 CACHED

#14 [third-consumer stage-1 3/3] COPY --from=build --chown=65532:65532
/out/data /data
#14 CACHED

#15 [third-consumer] exporting to image
#15 exporting layers done
#15 writing image
sha256:aa3e63cff54515965f77ac94c4bc4a1e3124f492ab4d50b59c2063e9695e0d92 done
#15 naming to docker.io/library/aat-report-revision-third-consumer 0.0s done
#15 DONE 0.1s
 Volume "aat-report-revision_third-consumer-data"  Creating
 Volume "aat-report-revision_third-consumer-data"  Created
 Container aat-report-revision-third-consumer-1  Creating
 Container aat-report-revision-third-consumer-1  Created
 Container aat-report-revision-third-consumer-1  Starting
 Container aat-report-revision-third-consumer-1  Started
 Container aat-report-revision-third-consumer-1  Waiting
 Container aat-report-revision-third-consumer-1  Healthy
```

Gambar 8.7: Menjalankan consumer tambahan melalui profile demo.

Pada sesi tersebut, snapshot monitoring menunjukkan durable baru telah mencapai ACK 192, sama dengan posisi stream dan kedua durable lainnya. Karena consumer baru memakai DeliverAll, pesan yang masih tersimpan sebelum ia dibuat juga tersedia untuk dibaca; producer tidak perlu mengirim ulang secara khusus kepadanya.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>          consumers: [.consumer_detail[]
>            | {name, ack: .ack_floor.stream_seq,
>               pending: .num_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 192,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 192,
      "pending": 0
    },
    {
      "name": "notifier-v1",
      "ack": 192,
      "pending": 0
    },
    {
      "name": "third-consumer-v1",
      "ack": 192,
      "pending": 0
    }
  ]
}
```

Gambar 8.8: Durable baru membaca stream yang sama sampai posisi ACK yang ditampilkan broker.

Perbandingan kondisi Aggregator sebelum dan sesudah menghasilkan file yang identik. Inspeksi Third Consumer juga menunjukkan inbox 193 dengan quarantine nol. Jumlah inbox diambil setelah snapshot broker, ketika stream masih menerima event. Dengan demikian, penambahan consumer berhasil tanpa perubahan container producer, dan consumer baru berhasil menyimpan hasil pengolahannya.

```
$ docker inspect $(docker compose ps -q aggregator) \
>   | jq -S '.[0] | {id: .Id, started_at: .State.StartedAt,
>       restarts: .RestartCount}' > /tmp/report-session/producer-after.json
> diff -s /tmp/report-session/producer-before.json \
>   /tmp/report-session/producer-after.json
> docker compose --profile demo exec -T third-consumer \
>   /notifier-consumer inspect | jq '{inbox_count, quarantine_count}'
>
Files /tmp/report-session/producer-before.json and /tmp/report-
session/producer-after.json are identical
{
  "inbox_count": 193,
  "quarantine_count": 0
}
```

Gambar 8.9: Producer tidak berubah dan consumer tambahan memiliki inbox sendiri.

Kriteria	Bukti
P5.1	Publish relay ke subject broker pada kode dan kenaikan sequence ketika Dashboard mati.
P5.2	Posisi durable terpisah pada Gambar 8.1 dan inbox masing-masing pada Gambar 8.6.
P5.3	Sequence 311 sebagai target gangguan, lalu ACK Dashboard 318 pada Gambar 8.3 dan 8.5.
P5.4	Startup Third Consumer, posisi ACK, dan kondisi producer pada Gambar 8.7–8.9.
P5.5	Pilihan fan-out, at-least-once, dan transaksi inbox idempoten dijelaskan pada bab ini.

8.4. Batasan

Consumer dapat mengejar backlog selama event yang dibutuhkannya masih tersedia di stream. Durable mempertahankan posisi baca, tetapi tidak memperpanjang retention. Jika consumer berhenti melewati batas retention, event yang sudah kedaluwarsa tidak dapat diambil kembali dari broker.

Konfigurasi satu replica juga belum menyediakan failover antarmesin. Di sisi consumer, idempotensi bergantung pada riwayat inbox yang tersimpan di volume lokal. Jika volume tersebut dihapus, consumer kehilangan catatan pesan yang pernah diolah. Retention broker dan keberlangsungan volume karena itu menjadi bagian dari batas jaminan pengiriman pada M1.

Kesimpulan

Pada profil M1 yang diuji, platform dapat menyatukan data BMKG dan PVMBG ke dalam HazardEvent, menerima tambahan atribut tanpa restart atau migrasi, serta membatasi data sesuai identitas client. Ketika PVMBG mengalami outage, API tetap menyediakan data terakhir dengan penanda stale, sementara data BMKG tetap dapat diperbarui. Pemulihan sumber kemudian diikuti polling tanpa menjalankan ulang jalur baca.

Pemisahan consumer juga terbukti pada pengujian fan-out. Dashboard dapat dihentikan atau dibangun ulang ketika producer dan Notifier tetap berjalan, lalu melanjutkan pembacaan event yang tersimpan di broker. Consumer baru dapat membaca stream yang sama tanpa perubahan producer.

Pada load test 5 Oktober 2026 dengan 60 worker dan delay PVMBG 3 detik, p95 BMKG sebesar 124,52 ms, throughput 980,26 respons sukses per detik, dan error selain 429 sebesar 0,034%. Sampel TCP menunjukkan sedikitnya 50 koneksi selama 89,00 detik. Hasil ini memenuhi ambang P2 pada profil beban yang diuji.

Ownership storage, kontrak event, outbox, dan consumer idempoten menjadi dasar untuk pengembangan milestone berikutnya. Jika jumlah instance ditambah, polling dan relay perlu dikoordinasikan, sedangkan state Auth serta consumer perlu dikelola agar sesuai dengan pola deployment tersebut. Pengujian kapasitas dan kegagalan juga harus diulang; hasil satu host pada M1 belum membuktikan high availability pada beberapa node.

Prioritas lanjutan adalah menelusuri kegagalan perubahan schema pada volume besar, mengurangi biaya snapshot yang terus tumbuh, dan menetapkan pembersihan data lokal. Sinkronisasi waktu perlu dijaga untuk expiry token serta freshness. Sebelum deployment di luar demo lokal, sistem juga membutuhkan TLS, pemisahan credential broker, dan mekanisme rotasi key.

Pembagian Tugas

Anggota	Perancangan	Implementasi	Penulisan laporan
M Hazim R Prajoda 13523009	Kontrak sumber, aturan pemetaan, Canonical Store	Mock BMKG/PVMBG, Aggregator, polling dan penyimpanan	P1 dan aspek flexible storage P4
Adhimas Aryo Bimo 13523052	Identitas client, scope, token, isolasi request	Auth/Token Service, Client API, ketahanan dan load test	P2, P3, serta pengukuran API
Refki Alfarizi 13523002	Event fan-out, durable consumer, jaringan dan integrasi	JetStream, consumer, orchestration, trial integrasi dan demo console	P5, container P4, penyusunan dan penggabungan laporan

Tabel 10.1: Kontribusi anggota pada perancangan, implementasi, dan penulisan laporan.

Masing-masing anggota menangani bagian sesuai tabel, sedangkan kontrak antarservice dan integrasi dibahas bersama agar implementasi dapat dihubungkan.

Referensi

[1] Tim Pengajar IF4031. *Milestone 1 Specification IF4031 – Tugas Besar – M1*. Spesifikasi tugas yang diberikan pada mata kuliah IF4031, 2026.

[2] Tim Pengajar IF4031. *IF4031 – Tugas Besar – Dokumen Terpusat*. Deskripsi sistem dan lingkup milestone, 2026.

[3] PostgreSQL Global Development Group. *PostgreSQL 16: JSON Types*. `https://www.postgresql.org/docs/16/datatype-json.html`.

[4] NATS Authors. *What is NATS?* dan *JetStream*. Dokumentasi resmi: `https://docs.nats.io/concepts/what-is-nats`; `https://docs.nats.io/learn/jetstream/`. Konfigurasi proyek mengacu pada NATS 2.10 dan kode repo.

[5] NATS Authors. *JetStream: Retention policies*. `https://docs.nats.io/learn/jetstream/retention-policies`.

[6] Go Authors. *Effective Go: Concurrency*. `https://go.dev/doc/effective_go#concurrency`.

[7] Apache Software Foundation. *Apache Kafka: Introduction*. `https://kafka.apache.org/documentation/`.

[8] RabbitMQ Authors. *Publish/Subscribe* dan *Streams and Superstreams*. `https://www.rabbitmq.com/tutorials/tutorial-three-go`; `https://www.rabbitmq.com/docs/streams`.

[9] SQLite Authors. *SQLite Is Serverless* dan *Appropriate Uses For SQLite*. `https://sqlite.org/serverless.html`; `https://sqlite.org/whentouse.html`.

[10] Docker Inc. *Docker Compose*. `https://docs.docker.com/compose/`.

[11] JSON Schema. *What is JSON Schema?* `https://json-schema.org/overview/what-is-jsonschema`.

[12] M. Jones, J. Bradley, N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. Mei 2015. `https://www.rfc-editor.org/rfc/rfc7519`.

[13] Kelompok DikejarDeadline. *Repositori implementasi IF4031 M1*. Commit `071b32d`; kode, konfigurasi, dan script pengujian yang dirujuk dalam Lampiran A. Trial integrasi dan load test dikumpulkan pada 5 Oktober 2026; command langsung untuk tangkapan terminal dijalankan pada 7 Oktober 2026.

Penelusuran Kode dan Bukti

A.1. Lokasi implementasi

Path berikut relatif terhadap root repo. Kode menjelaskan bagaimana mekanisme bekerja, sedangkan berkas evidence menyimpan hasil yang diamati ketika mekanisme tersebut diuji.

P1: pemetaan dan perubahan schema
`services/aggregator/internal/mapper/`, `services/aggregator/internal/poller/`, dan `services/pvmbg-mock/internal/mock/`.

P2: jalur baca dan isolasi
`services/client-api/internal/api/handler.go`, `services/aggregator/internal/store/store.go`, serta `scripts/load/`.

P3: autentikasi dan token
`services/auth/internal/auth/`, `services/client-api/internal/api/handler.go`, `.env.example`, `.gitignore`, dan `.dockerignore`.

P4: ownership dan dokumen
`docker-compose.yml`, Dockerfile tiap service, `services/aggregator/internal/store/migrations/`, serta perintah audit pada `services/aggregator/cmd/server/`.

P5: publish dan consumer
`services/aggregator/internal/outbox/`, `services/aggregator/internal/broker/`, `pkg/jetstreamconsumer/`, serta direktori `internal/store` pada kedua consumer.

A.2. Berkas evidence

Berkas berikut berada di `docs/report/evidence/`. Trial integrasi dan load test dilakukan pada 5 Oktober 2026. Pengujian fungsional melalui HTTP dan Docker dilakukan pada 7 Oktober. Waktu, command, exit code, dan output pengujian dicatat dalam `terminal-commands.json`. Contoh record, daftar migrasi, dan kondisi container juga tersedia pada berkas audit.

Berkas	Isi
`integration.json`, `integration.log`	Pemetaan, schema, outage, token, rebuild, dan fan-out; 20 catatan pemeriksaan, trial lulus.
`validated-records.log`	Validasi delapan record dari trial integrasi dan command langsung, termasuk dua pasang revisi PVMBG.
`contract-tests.log`	Test schema, mapper, scope API, dan validasi token.
`access-and-networks.json`	Empat request credential, akses Media, dan anggota network setiap container.
`load.json`, `load.log`, `load-statuses.json`	Konfigurasi, throughput, percentile, status error, dan refresh saat beban.
`tcp.json`, `tcp.log`	Sampel koneksi TCP established selama beban.
`pvmbg-delay.json`	Delay yang diminta dan latency sumber yang diukur.
`outage-recovery.json`	Status stale/fresh kedua sumber dan pemulihan tanpa restart.
`schema-volume-review.txt`	Catatan kegagalan perubahan schema pada sekitar 1000 record PVMBG.
`correlation.log`	Log JSON API dan Aggregator untuk request dengan correlation ID yang sama.
`terminal-commands.json`	Command langsung, waktu pengambilan, exit code, stdout, dan stderr untuk tangkapan terminal.
`mapping-command-records.json`, `schema-command-audit.json`	Record canonical dan output audit lengkap dari pengujian command 7 Oktober.
`rebuild-container-states.json`	ID, waktu mulai, dan restart count delapan container sebelum serta sesudah rebuild.
`environment.json`	Commit, versi alat, mesin, project, dan konfigurasi non-secret.
`assessment.json`	Pemeriksaan threshold dan interval koneksi dari data mentah.